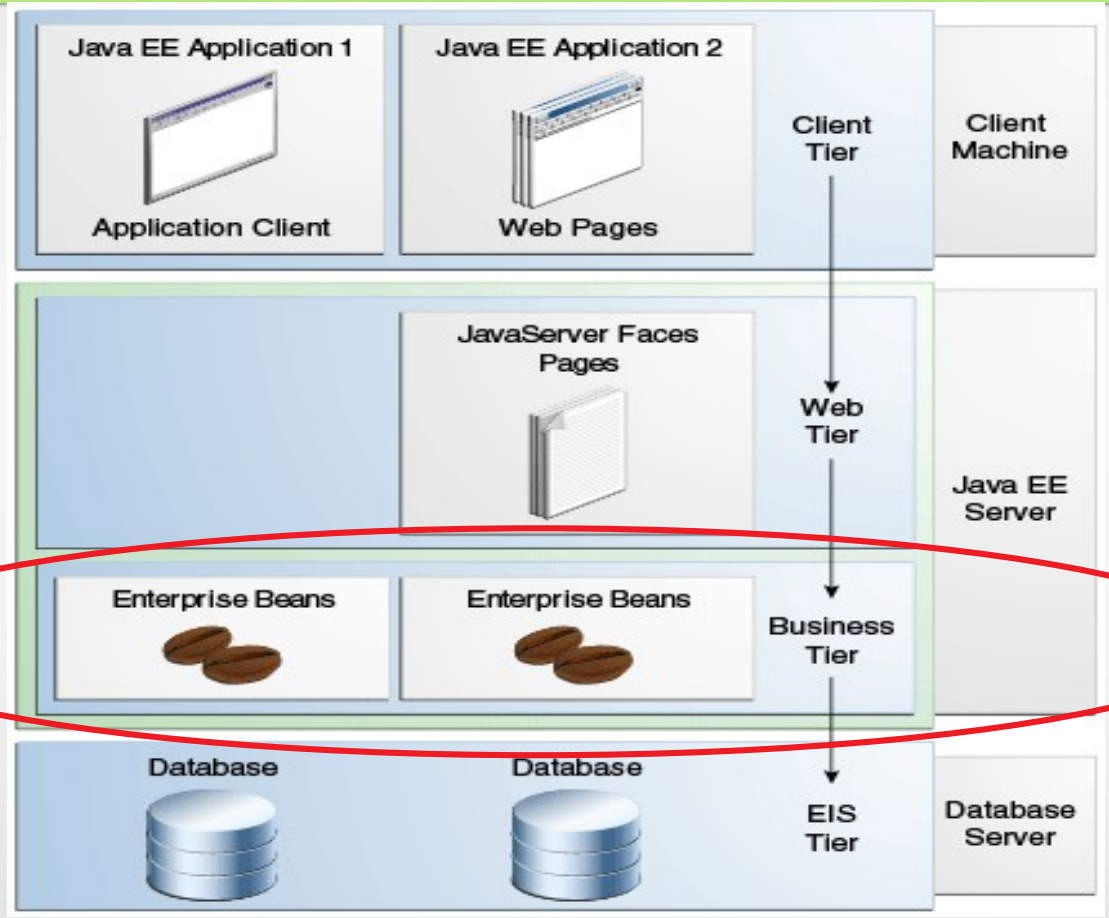
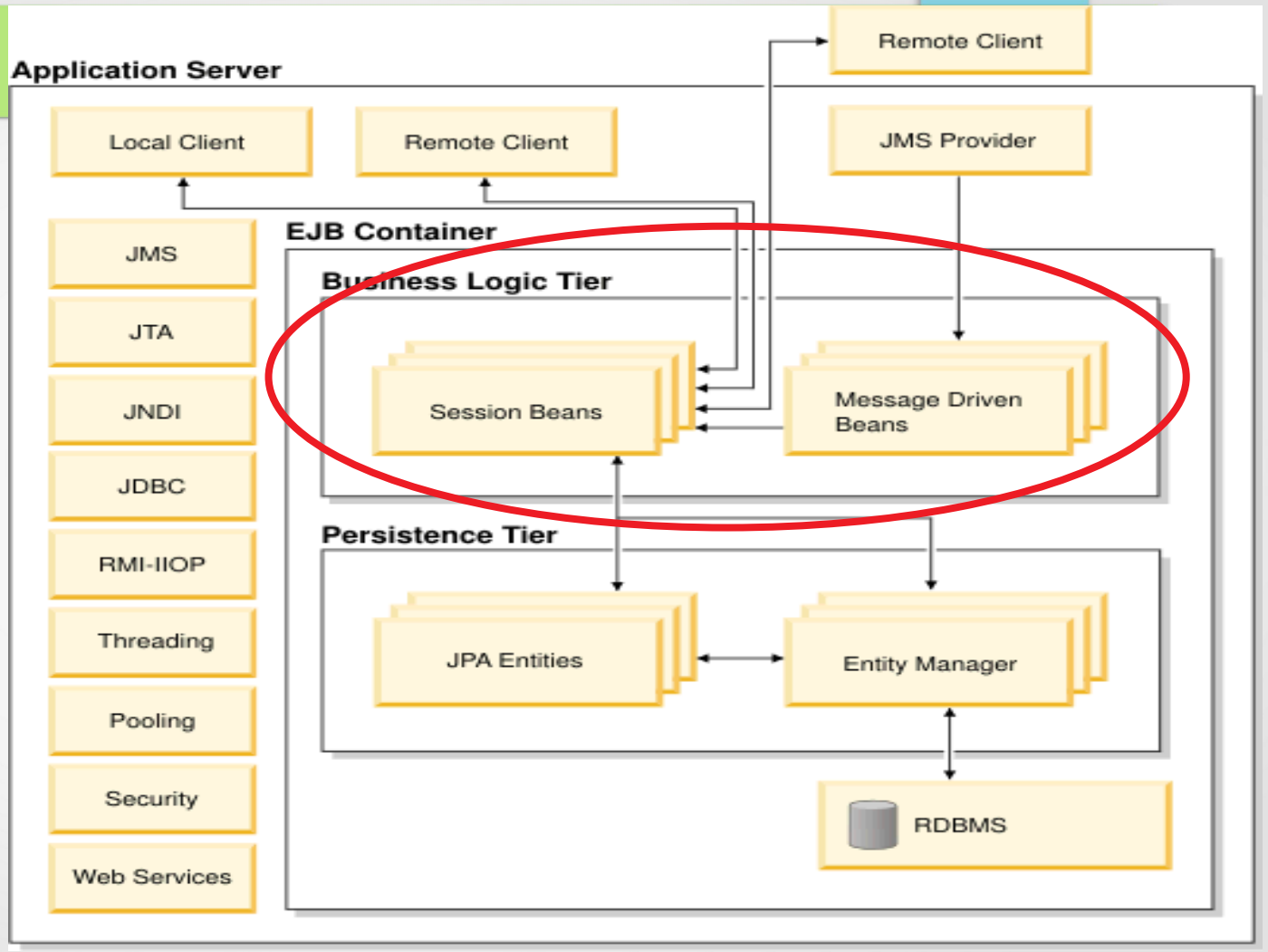
Enterprise JavaBeans (EJBs)

Cesar Julio Bustacara M.

Tier de Negocio



Tier de Negocio



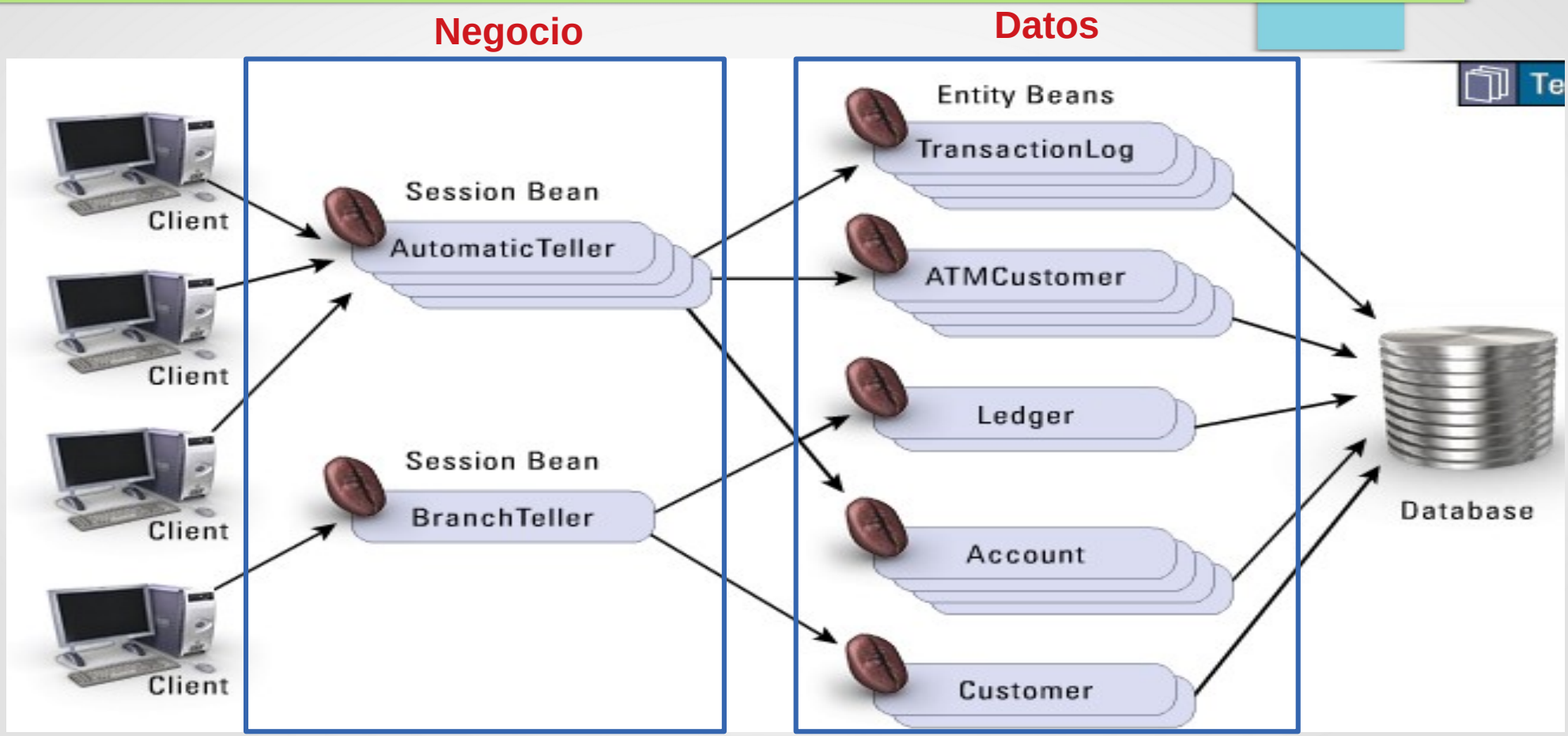
Definición

- Enterprise JavaBeans (EJBs)
  - Componentes de software que:
    - Encapsulan la lógica de negocio
    - Son reusables
    - Son portables
    - Se ejecutan por medio de un contenedor
  - Es una **clase Java pública**, que implementa los **métodos de negocio** e **implementa determinadas interfaces**.

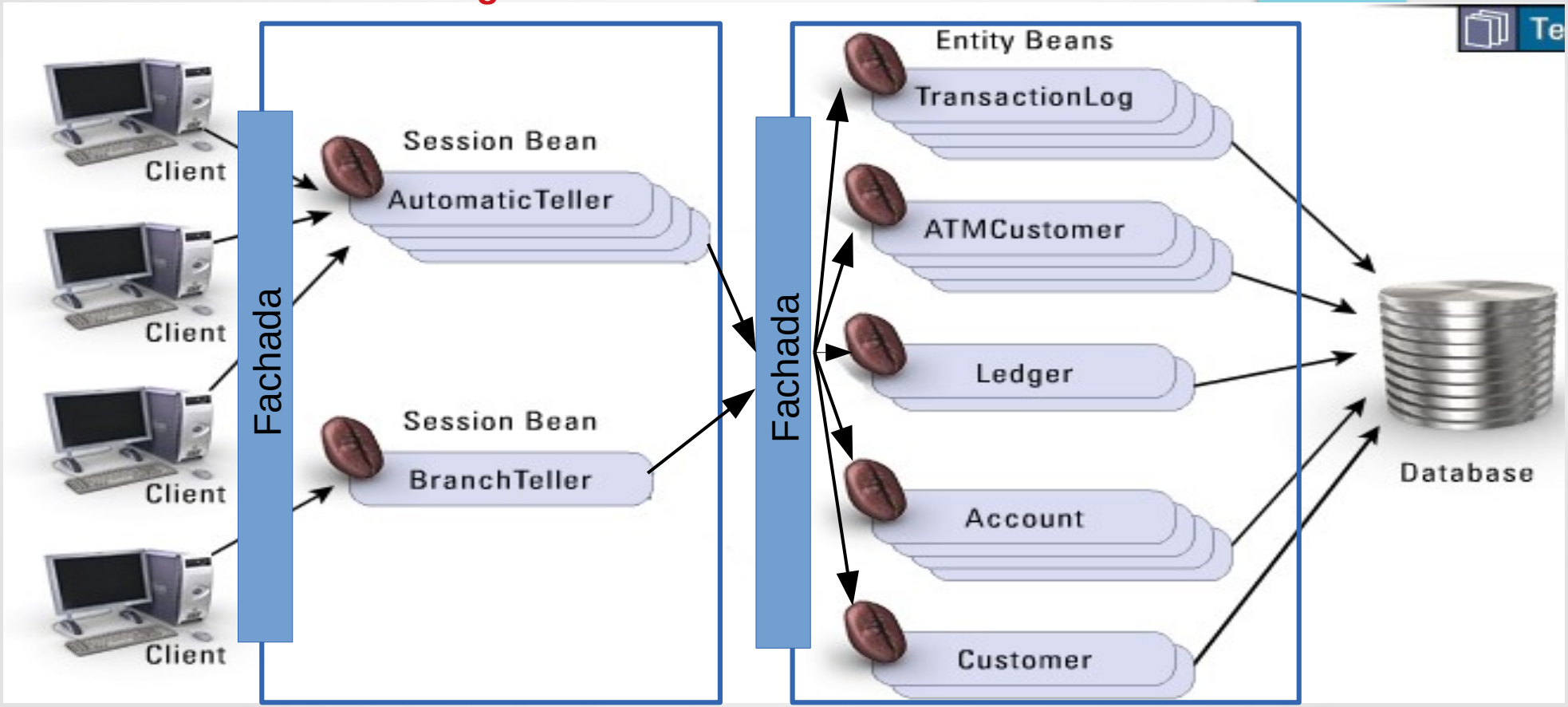
Tipos de EJBs

- Para cumplir con la funcionalidad de la lógica de negocio, JEE provee los siguientes tipos de EJBs:
  - **Sesión**: Ejecuta una tarea para un cliente. Opcionalmente, puede implementar servicios web
  - **Mensajeria**: Actua como un listener para un tipo específico de mensajes.

Tier de Negocio



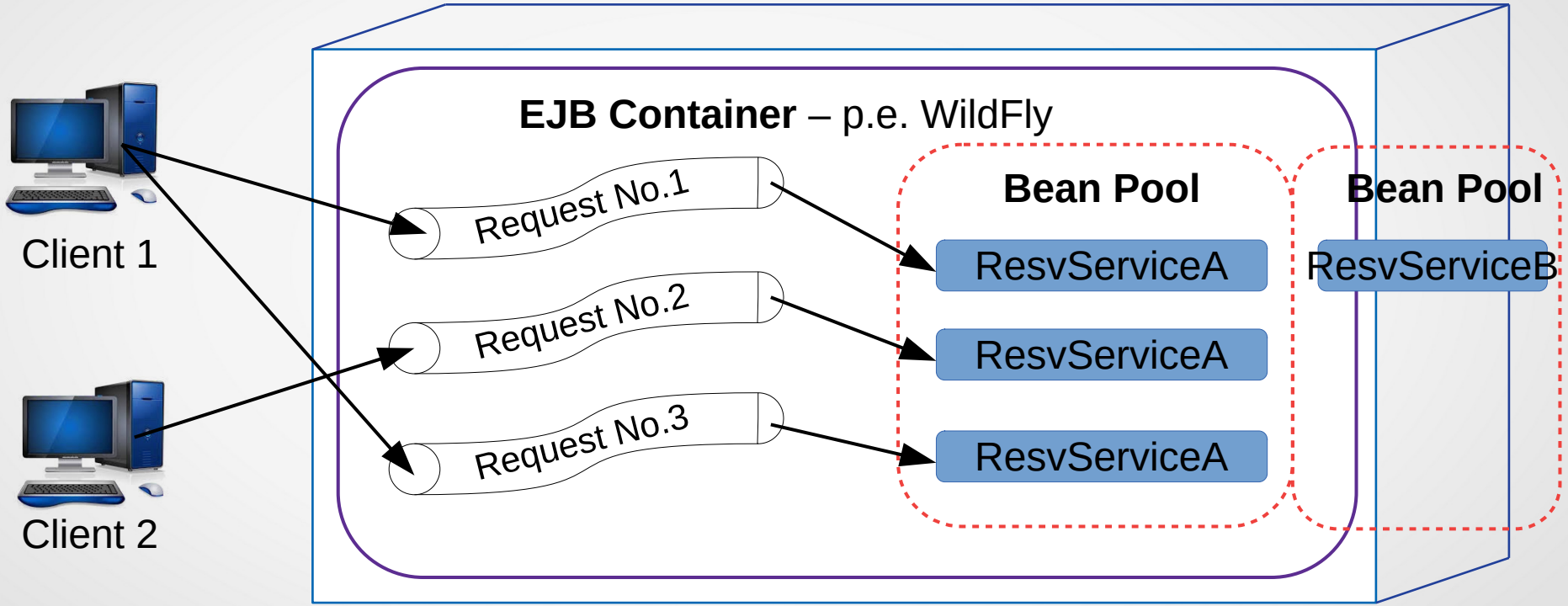
Tier de Negocio – bajo acoplamiento



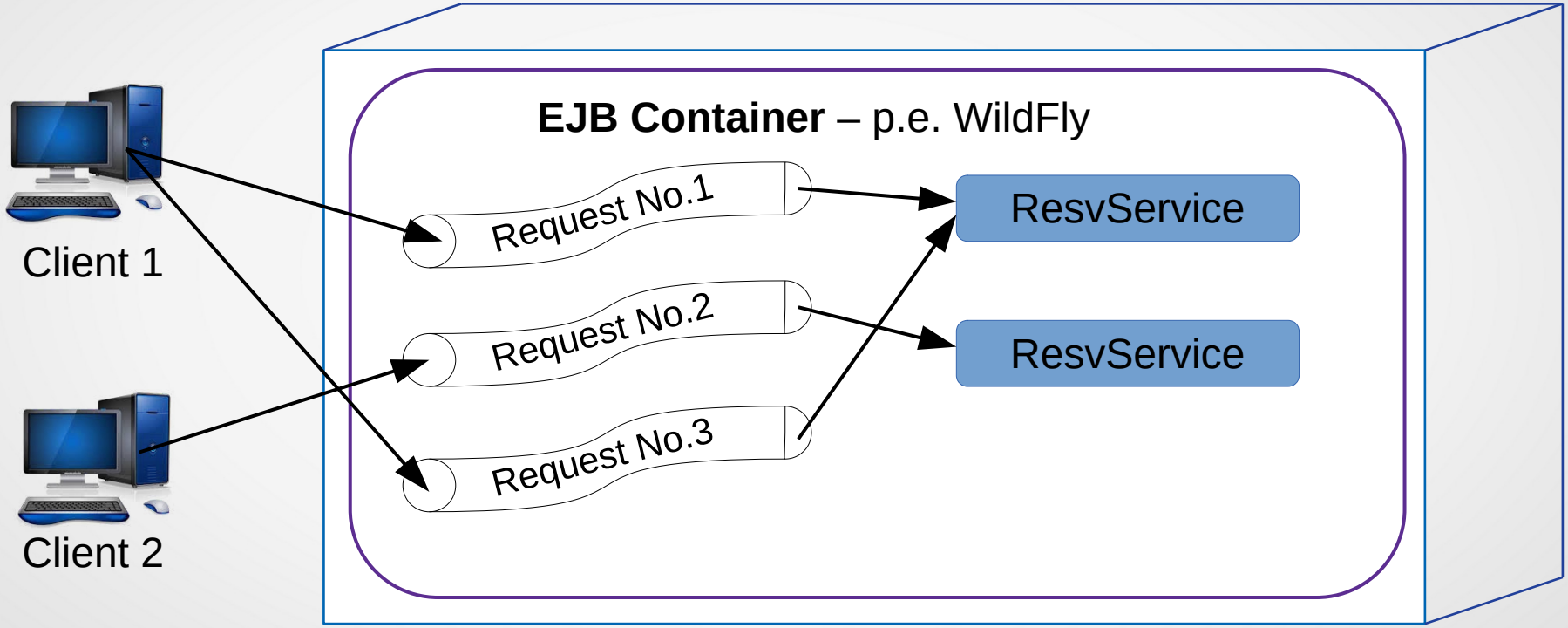
Tipos de EJBs de Sesión

- **Stateful**: mantiene en el servidor el estado de la sesión del cliente
- **Stateless**: no mantiene estado
- **Singleton**: existe un sólo bean para toda la aplicación o sea para todos los clientes por lo que soporta acceso concurrente

EJB de Sesión - Stateless



EJB de Sesión - Stateful



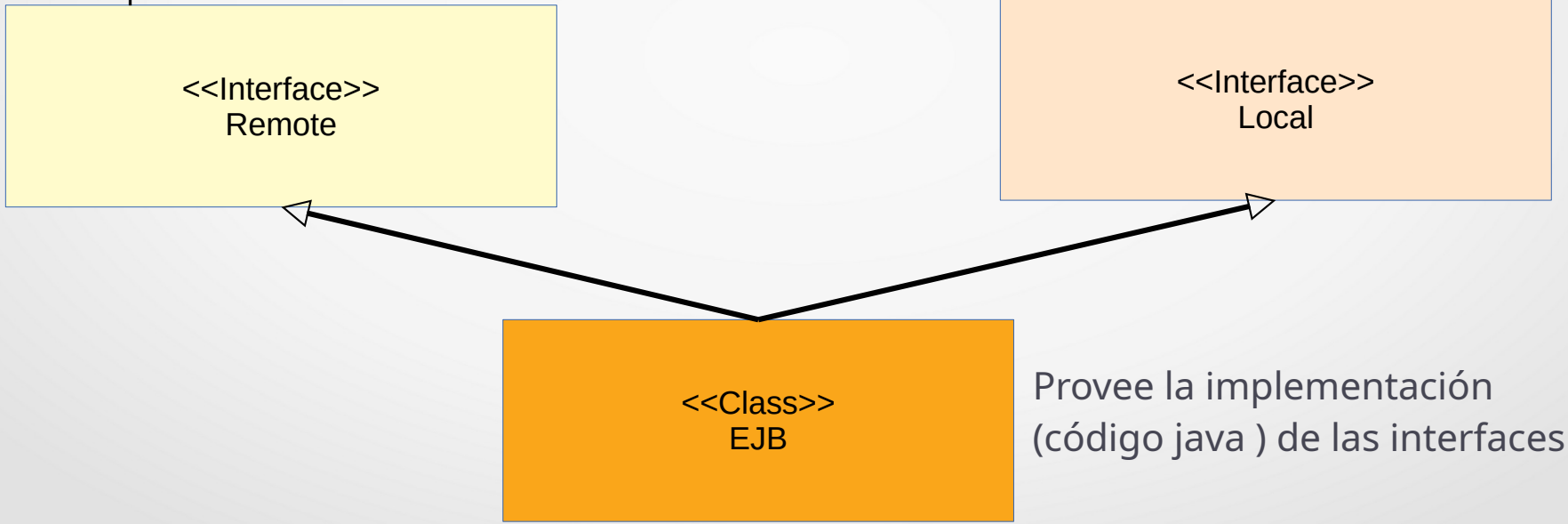
Elementos de los EJB de Sesión

Define los servicios públicos a los clientes remotos

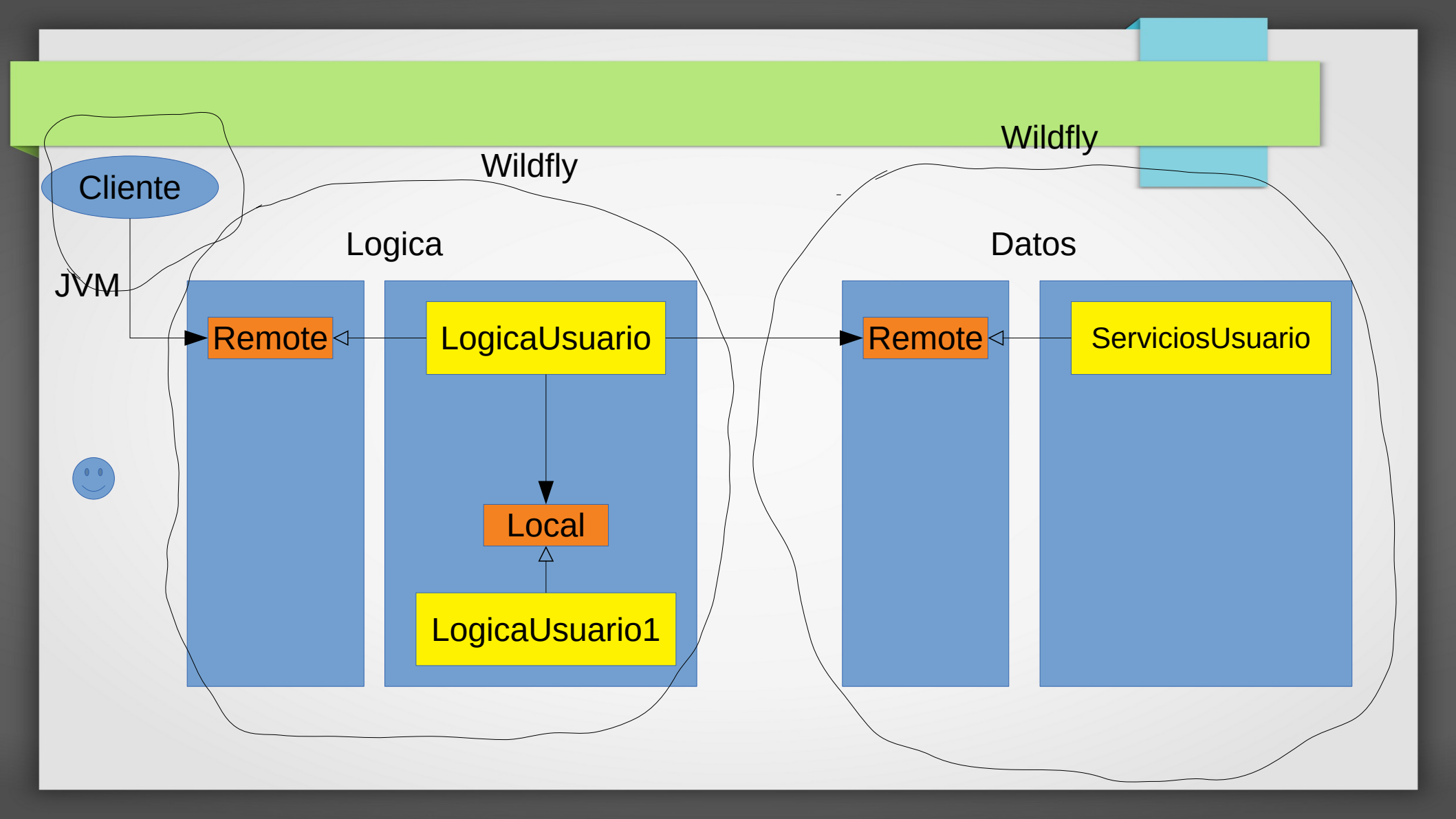
- Clientes en máquinas diferentes
- Clientes en JVM diferentes
- Paso por valor

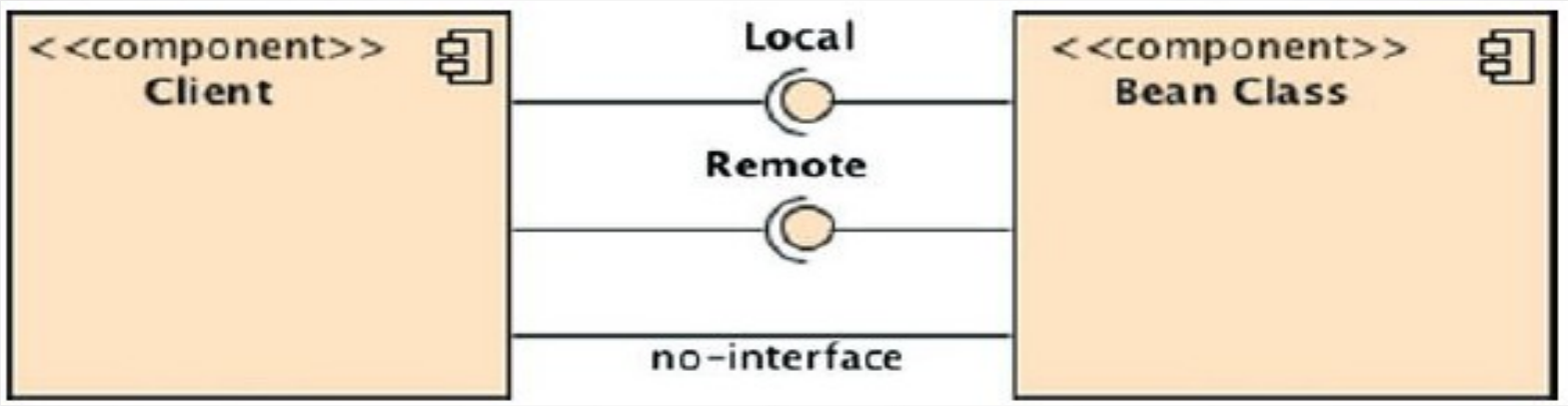
Define los servicios públicos a los clientes locales

- Clientes en la misma JVM
- Paso por referencia



Provee la implementación (código java) de las interfaces


Wildfly
Wildfly
Cliente
JVM
Logica
Datos
Remote
LogicaUsuario
Remote
ServiciosUsuario
Local
LogicaUsuario1



- La vista de “**no-interface**” es una variación de la vista local que expone todos los métodos públicos de negocio de manera local sin exponer interfaces.

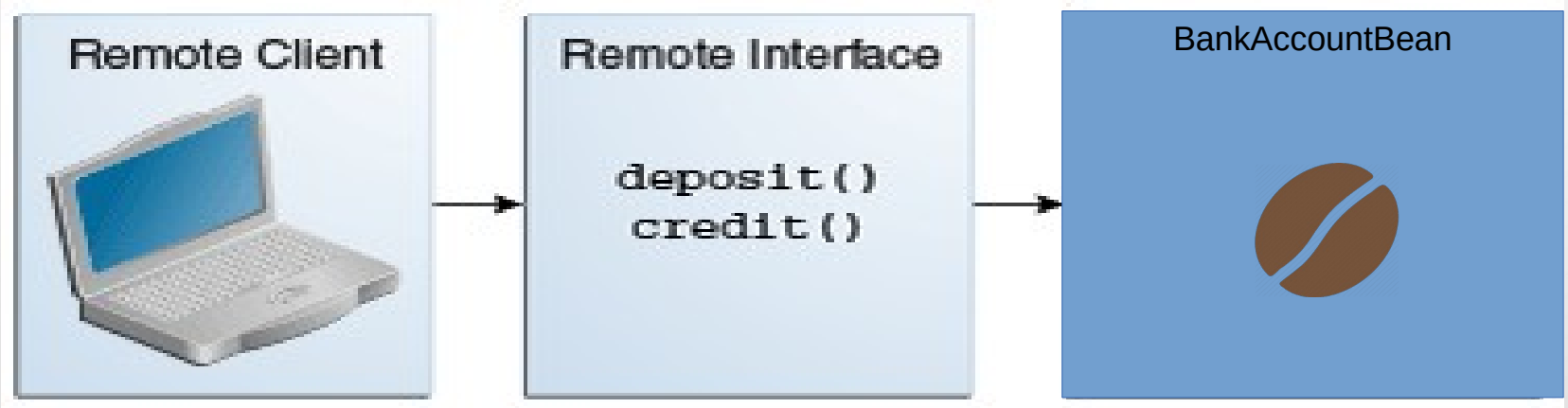
- Los clientes que realizan llamadas remotas usan **RMI o RMI-IIOP**

Ejemplo interface Remote

@ expone interface remota.

```
@Remote
 public interface InterfaceName {
    ...
 }
```

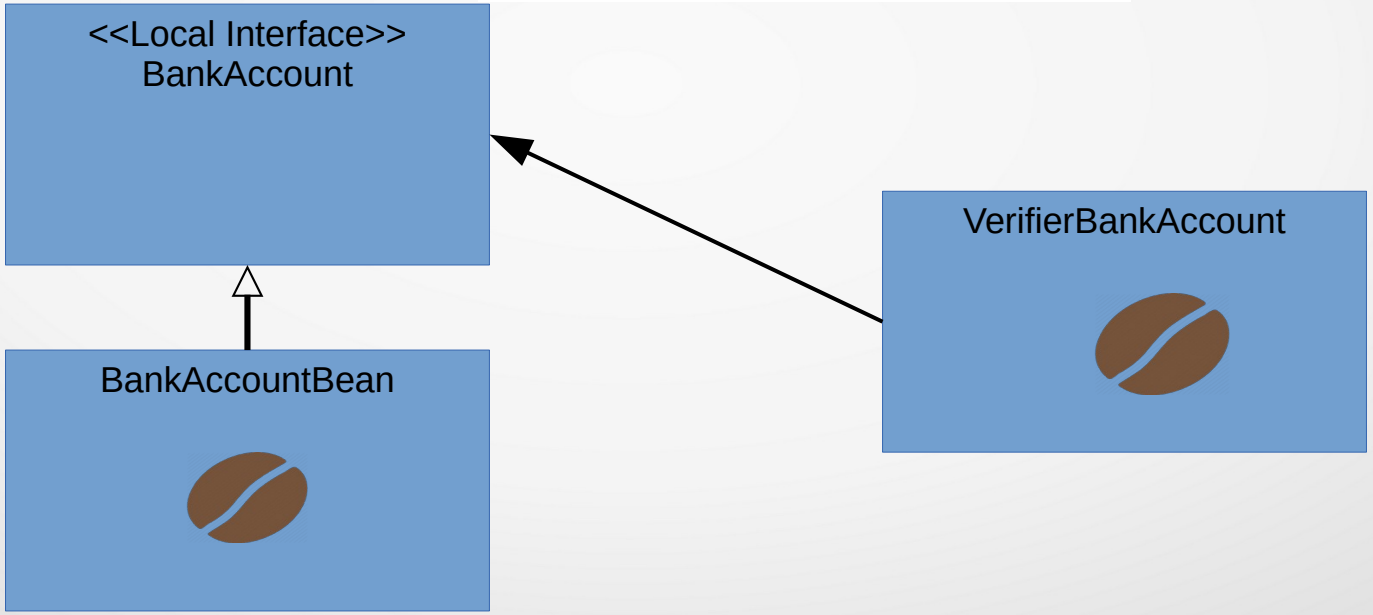
@ clientes en diferentes VM ó en diferentes nodos



@ clientes en misma VM

@ expone interface local.

```
@Local
public interface InterfaceName {
...
}
```



Ejemplo de un EJB de Sesión de tipo stateless

Interface Remote

```
import jakarta.ejb.Remote;

@Remote
public interface ServicioEJBRemote {

   public String metodoServicio();
}
```

Interface Local

```
import jakarta.ejb.Local;


/**
 * This is the business interface for ServicioEJB enterprise bean.
 */

@Local
public interface ServicioEJBLocal {

}
```

Implementacion del EJB

```java
import jakarta.ejb.Stateless;

/**
 * Session Bean implementation class ServicioEJB
 */
@Stateless
public class ServicioEJB implements ServicioEJBRemote {

    /**
     * Default constructor.
     */
    public ServicioEJB() {
        // TODO Auto-generated constructor stub
    }

    @Override
    public String metodoServicio() {
        // TODO Auto-generated method stub
        return "Servicio Remoto Invocado";
    }

}
```

Aplicación cliente

```
public class Principal {
     public static void main(String[] args) {

        Properties jndiProperties = new Properties();
        jndiProperties.put(Context.INITIAL_CONTEXT_FACTORY, "org.wildfly.naming.client.WildFlyInitialContextFactory");
        jndiProperties.put(Context.PROVIDER_URL, "http-remoting://localhost:8080");
        jndiProperties.put("jboss.naming.client.ejb.context", true);

        Context ctx = null;

          try {
               ctx = new InitialContext(jndiProperties);

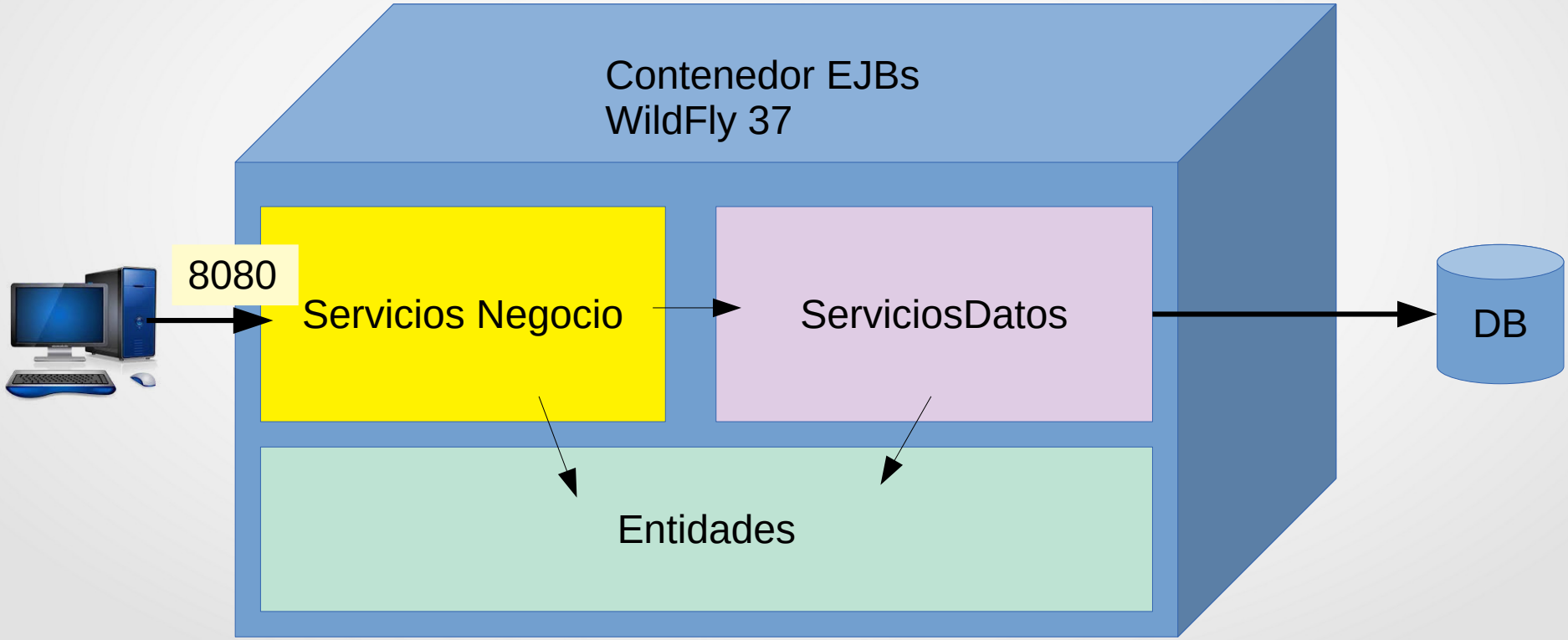
               ServicioEJBRemote xxx = (ServicioEJBRemote) ctx.lookup("ejb:Demo02EAR/Demo02/ServicioEJB!
logica.ServicioEJBRemote");

               String resultado = xxx.metodoServicio();
               System.out.println(resultado);

          } catch (NamingException e) {
               // TODO Auto-generated catch block
               e.printStackTrace();
          }
     }
}
```

Ejercicio

A) Crear una aplicación de 2-niveles para manejar usuarios y autenticación



Ejercicio

B) Crear una aplicación de 3-niveles para manejar usuarios y autenticación

